



Section 8 / 24

GNU Debugger (GDB)

Debugging is an important skill to learn for developers and pentesters alike. Debugging is a term used for **finding and removing issues** (i.e., bugs) from our code, **hence the name de-bugging**. When we develop a program, we will very frequently run into bugs in our code. It is not efficient to keep changing our code until it does what we expect of it. Instead, we perform debugging by setting breakpoints and seeing how our program acts on each of them and how our input changes between them, which should give us a clear idea of what is causing the **bug**.

Programs written in high-level languages can set breakpoints on specific lines and run the program through a debugger to monitor how they act. With Assembly, we deal with machine code represented as assembly instructions, so our breakpoints are set in the memory location in which our machine code is loaded, as we will see.

To debug our binaries, we will be using a well-known debugger for Linux programs called **GNU Debugger** (**GDB**). There are other similar debuggers for Linux, like **Radare** and **Hopper**, and for Windows, like **Immunity Debugger** and **WinGDB**. There are also powerful debuggers available for many platforms, like **IDA Pro** and **EDB**. In this module, we will be using GDB. It is the most reliable for Linux binaries since it is built and maintained directly by GNU, which gives it an excellent integration with the Linux system and its components.

Installation

GDB is installed in many Linux distributions, and it is also installed by default in Parrot OS and PwnBox. In case it is not installed in your VM, you can use `apt` to install it with the following commands:

```
shellsession
kimvutrinh@htb[/htb]$ sudo apt-get update
kimvutrinh@htb[/htb]$ sudo apt-get install gdb
```

One of the great features of `GDB` is its support for third-party plugins. An excellent plugin that is well maintained and has good documentation is `GEF`. GEF is a free and open-source GDB plugin that is built precisely for reverse engineering and binary exploitation. This fact makes it a great tool to learn.

To add GEF to GDB, we can use the following commands:

```
shellsession
kimvutrinh@htb[/htb]$ wget -O ~/.gdbinit-gef.py -q https://gef.blah.cat/py
kimvutrinh@htb[/htb]$ echo source ~/.gdbinit-gef.py >> ~/.gdbinit
```

Getting Started

Now that we have both tools installed, we can run gdb to debug our `HelloWorld` binary using the following commands, and GEF will be loaded automatically:

```
shellsession
```

```
kimvutrinh@htb[/htb]$ gdb -q ./helloWorld
...SNIP...
gef➤
```

As we can see from `gef➤`, GEF is loaded when GDB is run. If you ever run into any issues with `GEF`, you can consult with the [GEF Documentation](#), and you will likely find a solution.

Going forward, we will frequently be assembling and linking our assembly code and then running it with `gdb`. To do so quickly, we can use the `assembler.sh` script we wrote in the previous section with the `-g` flag. It will assemble and link the code, and then run it with `gdb`, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ ./assembler.sh helloWorld.s -g
...SNIP...
gef➤
```

Info

Once `GDB` is started, we can use the `info` command to view general information about the program, like its functions or variables.

Tip: If we want to understand how any command runs within `GDB`, we can use the `help CMD` command to get its documentation. For example, we can try executing `help info`

Functions

To start, we will use the `info` command to check which `functions` are defined within the binary:

```
gef> info functions

All defined functions:

Non-debugging symbols:
0x0000000000401000  _start
```

shellsession

As we can see, we found our main `_start` function.

Variables

We can also use the `info variables` command to view all available variables within the program:

```
gef> info variables

All defined variables:

Non-debugging symbols:
0x0000000000402000  message
0x0000000000402012  __bss_start
0x0000000000402012  _edata
0x0000000000402018  _end
```

shellsession

As we can see, we find the `message`, along with some other default variables that define memory segments. We can do many things with functions, but we will focus on two main points: Disassembly and Breakpoints.

Disassemble

To view the instructions within a specific function, we can use the `disassemble` or `disas` command along with the function name, as follows:

```
shellsession
gef> disas _start

Dump of assembler code for function _start:
   0x0000000000401000 <+0>: mov     eax,0x1
   0x0000000000401005 <+5>: mov     edi,0x1
   0x000000000040100a <+10>: movabs  rsi,0x402000
   0x0000000000401014 <+20>: mov     edx,0x12
   0x0000000000401019 <+25>: syscall
   0x000000000040101b <+27>: mov     eax,0x3c
   0x0000000000401020 <+32>: mov     edi,0x0
   0x0000000000401025 <+37>: syscall
End of assembler dump.
```

As we can see, the output we got closely resembles our assembly code and the disassembly output we got from `objdump` in the previous section. We need to focus on the main thing from this disassembly: the memory addresses for each instruction and operands (i.e., arguments).

Having the memory address is critical for examining the variables/operands and setting breakpoints for a certain instruction.

You may notice through debugging that some memory addresses are in the form of `0x0000000004xxxx` , rather than their raw address in memory `0xfffffffffaa8a25ff` . This is due to `$rip-relative addressing` in Position-Independent Executables `PIE` , in which the memory addresses are used relative to their distance from the instruction pointer `$rip` within the program's own Virtual RAM, rather than using raw memory addresses. This feature may be ^{a mistake???} `disabled to reduce` the risk of binary exploitation.

Next, let us go through the basics of debugging with GDB by using breakpoints, examining data, and stepping through the program.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline

← Previous

Section 8 / 24

XP +20

Next

Mark Complete & Next